ARTIFICIAL INTELLIGENCE • 23A31301T

# UNIT I

## Introduction and Intelligent Agents

Digital Notes | Diagrams, worked examples and exam tips

**SYLLABUS COVERED IN THIS UNIT**

AI problems, foundations and history of AI. Intelligent agents: agents and environments, the concept of rationality, the nature of environments, structure of agents, problem-solving agents, problem formulation.

## 1. What is Artificial Intelligence?

**Artificial Intelligence (AI)** is the branch of computer science that builds machines able to perform tasks which normally need human intelligence: understanding language, recognising images, reasoning, learning from experience and making decisions.

**DEFINITION**

AI is the science and engineering of making **intelligent agents**: systems that perceive their environment and take actions that maximise their chance of achieving their goals.

Different thinkers define 'intelligence' differently. Textbooks group the definitions into four approaches, shown below. Modern AI mostly follows the last one, **acting rationally**.

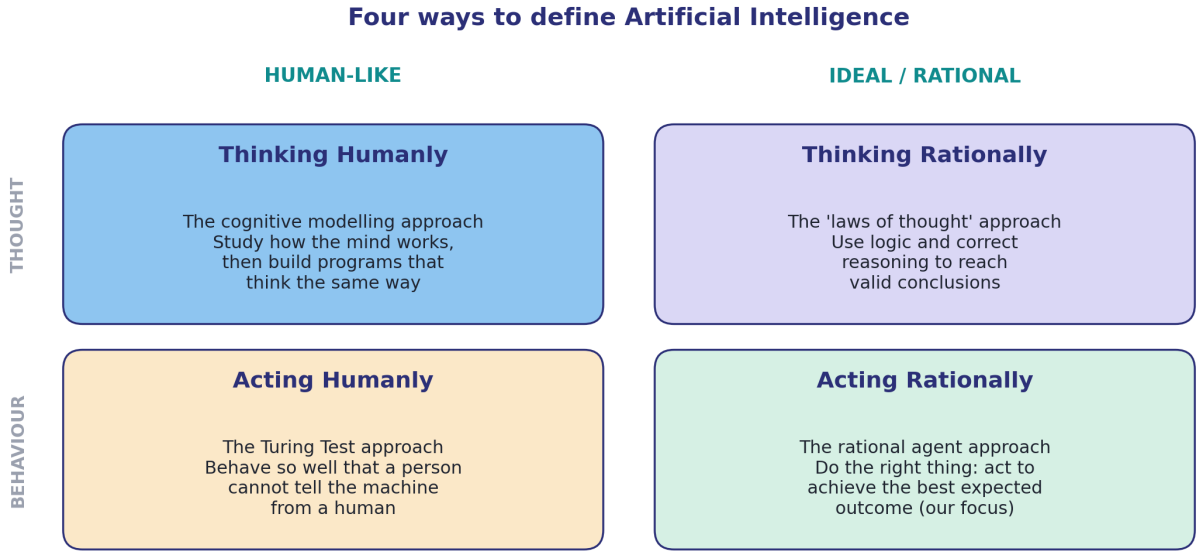



*Fig 1.1 The four approaches to AI (Russell & Norvig)*

| Approach | Key idea | Example / test |
|---|---|---|
| **Acting humanly** | Machine behaves like a person | Turing Test (1950): a judge chats with a human and a machine; if the judge cannot tell them apart, the machine passes |
| **Thinking humanly** | Machine reasons the way people do | Cognitive science, brain imaging, models such as GPS (General Problem Solver) |
| **Thinking rationally** | Machine follows laws of logic | Aristotle's syllogisms, logic-based AI systems |
| **Acting rationally** | Machine does the 'right thing' for its goal | Rational agents: a self-driving car, a chess engine, a recommender system |

**EXAM TIP**
The Turing Test needs these capabilities: **natural language processing, knowledge representation, automated reasoning, machine learning**. The *Total* Turing Test adds **computer vision and robotics**.

## 2. AI Problems

AI problems range from simple games to everyday human skills. Most are hard because the search space is huge, knowledge is incomplete and the world keeps changing.

| Task area | Examples | Nature of the problem |
|---|---|---|
| **Mundane tasks** | Perception (vision, speech), natural language, common-sense reasoning, robot control | Easy for humans, surprisingly hard for machines |
| **Formal tasks** | Games (chess, Go), mathematics (theorem proving, logic) | Clear rules, but the search space is enormous |
| **Expert tasks** | Medical diagnosis, engineering design, financial analysis, scientific discovery | Need deep specialist knowledge |

### Typical AI problem areas

- **Natural Language Processing (NLP):** translation, chatbots, summarisation.
- **Computer vision:** face recognition, medical imaging, self-driving cars.
- **Speech recognition:** voice assistants such as Siri and Alexa.
- **Planning and robotics:** warehouse robots, route planning, drones.
- **Expert systems:** rule-based advisors in medicine and industry (Unit V).
- **Machine learning:** systems that improve with data and experience (Unit IV).

**NOTE**
A good way to judge an AI problem is to ask four questions: **Can it be decomposed? Can steps be undone? Is the world predictable? Does it need a good solution or just any solution?**

# 3. Foundations of AI

AI did not appear from nowhere. It grew out of ideas from many older fields. Each field gave AI a key tool.

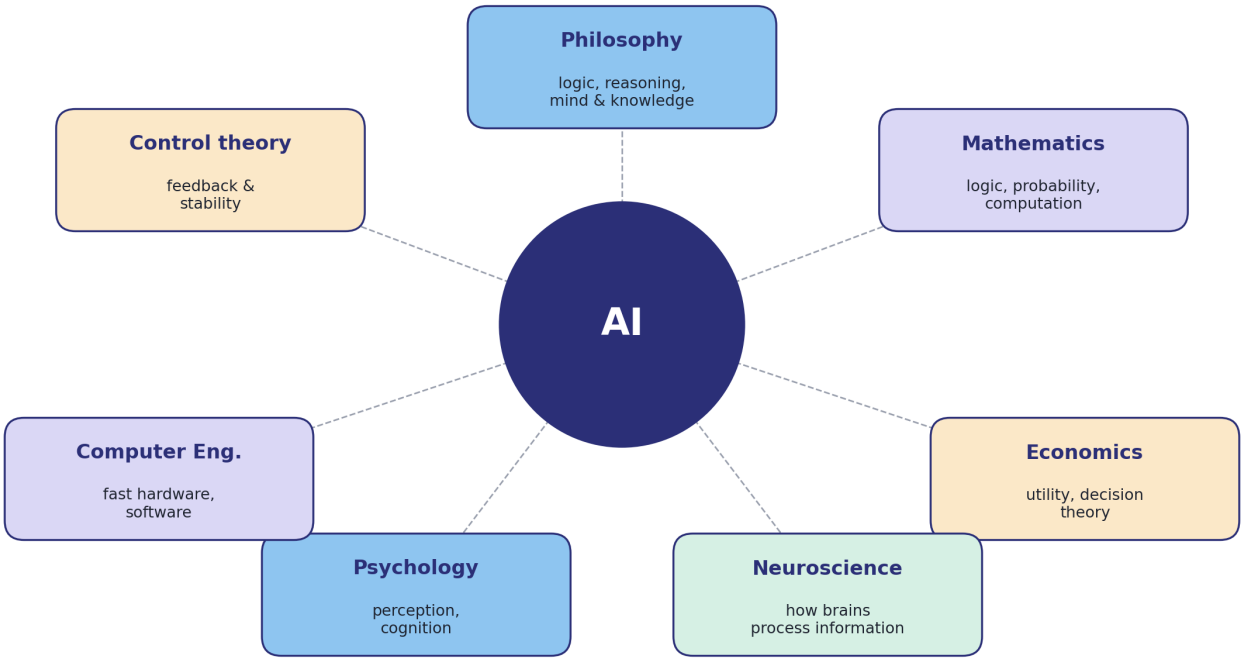



*Fig 1.3 Disciplines that contribute to AI (Linguistics: language & grammar, also contributes)*

| Discipline | What it gave AI |
|---|---|
| **Philosophy** | Formal logic, theories of reasoning, the mind-body question, the idea that thought can follow rules |
| **Mathematics** | Logic, algorithms, computability, probability and statistics, optimisation |
| **Economics** | Utility theory, decision theory, game theory, Markov decision processes |
| **Neuroscience** | How neurons process information; inspiration for artificial neural networks |
| **Psychology** | Models of perception, memory, learning and problem solving |
| **Computer engineering** | Fast and cheap hardware (CPUs, GPUs), programming languages, software tools |
| **Control theory** | Feedback loops, self-regulating machines that minimise error |
| **Linguistics** | Grammar and meaning, the basis of natural language processing |

# 4. History of AI

The story of AI is a cycle of big hopes, disappointments and fresh breakthroughs.

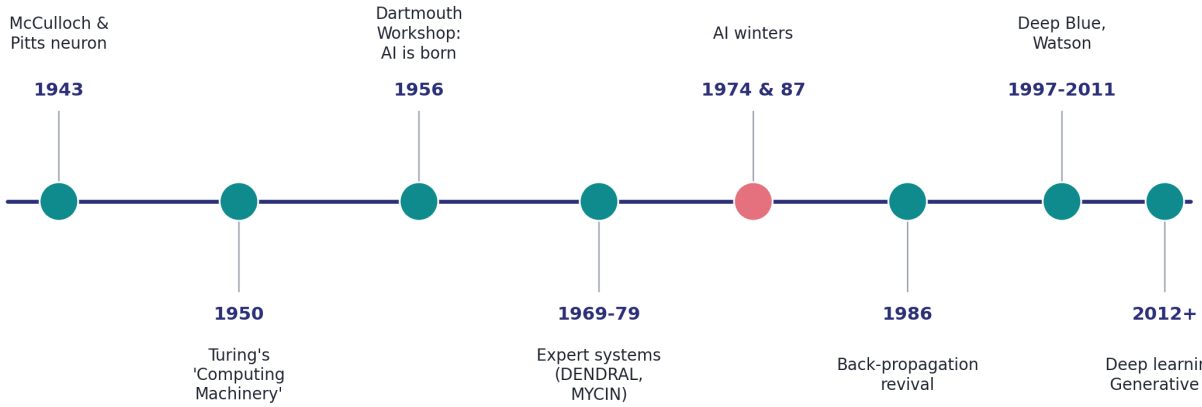



*Fig 1.2 Milestones in the history of AI*

| Period | Phase | What happened |
|---|---|---|
| 1943-1955 | Gestation | McCulloch-Pitts artificial neuron (1943), Hebbian learning, Turing's paper (1950), first neural-network computer |
| 1956 | Birth of AI | Dartmouth Workshop; John McCarthy coins the term 'Artificial Intelligence' |
| 1952-1969 | Early enthusiasm | Logic Theorist, General Problem Solver, LISP language (1958), checkers-playing program |
| 1966-1973 | A dose of reality | Programs failed to scale (combinatorial explosion), machine translation disappointed; funding cut |
| 1969-1986 | Expert systems | DENDRAL, MYCIN, XCON: knowledge-based systems succeed commercially |
| 1987-1993 | AI winter | Expert systems proved costly to maintain; the market collapsed |
| 1986-present | Neural nets and data | Back-propagation revival, probabilistic reasoning, machine learning, intelligent agents |
| 1997-2016 | Landmarks | Deep Blue beats Kasparov (1997), Watson wins Jeopardy! (2011), AlphaGo beats Lee Sedol (2016) |
| 2012-today | Deep learning era | Big data and GPUs power image, speech and language models and generative AI |

# 5. Intelligent Agents: Agents and Environments

**DEFINITION**
An **agent** is anything that **perceives** its environment through **sensors** and **acts** upon that environment through **actuators**.

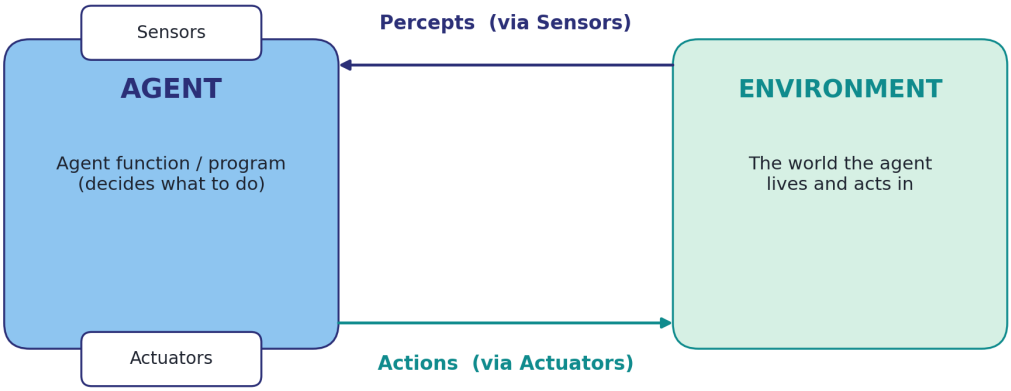



*Fig 1.4 An agent perceives its environment through sensors and acts on it through actuators*

| Agent | Sensors | Actuators |
|---|---|---|
| **Human** | Eyes, ears, skin, nose | Hands, legs, voice |
| **Robot** | Cameras, infrared, sonar, LIDAR | Motors, wheels, grippers |
| **Software agent** | Keystrokes, files, network packets | Screen display, writing files, sending messages |
| **Thermostat** | Temperature sensor | Heater switch |

## Key terms

- **Percept:** the agent's input at a single moment. **Percept sequence:** the complete history of everything it has perceived.
- **Agent function:** a mathematical mapping from every possible percept sequence to an action: f : P* → A.
- **Agent program:** the concrete code that implements the agent function on a physical architecture. *Agent = Architecture + Program.*

**Example: the vacuum-cleaner world.** The world has two squares, A and B. The robot senses its location and whether the square is dirty. It can move Left, move Right, or Suck.

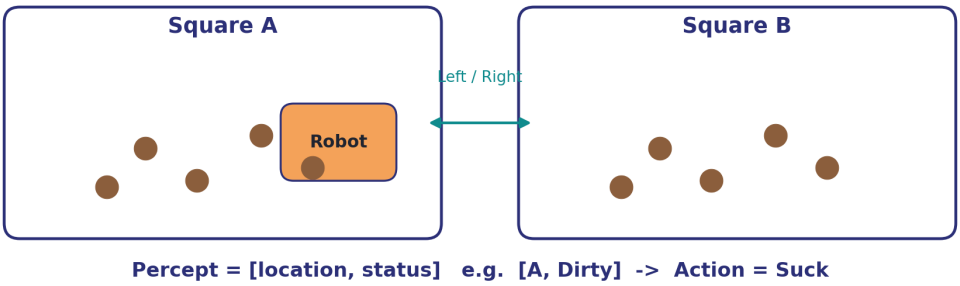



*Fig 1.5 The two-square vacuum-cleaner world, a classic toy environment*

| Percept sequence | Action |
|---|---|
| [A, Clean] | Right |
| [A, Dirty] | Suck |
| [B, Clean] | Left |
| [B, Dirty] | Suck |

# 6. The Concept of Rationality

**DEFINITION**
A **rational agent** is one that, for every possible percept sequence, selects an action that is **expected to maximise its performance measure**, given the evidence from the percept sequence and its built-in knowledge.

Rationality at any moment depends on four things:

- **Performance measure:** the criterion that defines success (for the vacuum robot: amount of dirt cleaned, electricity used, noise).
- **Prior knowledge:** what the agent already knows about the environment.
- **Actions available:** what the agent can actually do.
- **Percept sequence:** everything the agent has perceived so far.

**REMEMBER**
Rationality is **not** omniscience (knowing everything), **not** clairvoyance (predicting the future) and **not** perfection. A rational agent maximises *expected* performance, not actual performance. Example: crossing a street after looking both ways is rational even if a falling object hits you.

A rational agent should also **gather information** (exploration), **learn** from experience and be **autonomous**: depend on its own percepts rather than only on knowledge built in by the designer.

## Specifying the task: PEAS description

Before designing an agent, we specify its **task environment** using **PEAS**: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors.

| Agent | Performance | Environment | Actuators | Sensors |
|---|---|---|---|---|
| **Automated taxi** | Safe, fast, legal, comfortable trip, profit | Roads, traffic, pedestrians, customers | Steering, accelerator, brake, signal, horn | Cameras, GPS, speedometer, sonar, odometer |
| **Medical diagnosis system** | Healthy patient, low cost | Patient, hospital, staff | Display of questions, tests, diagnoses | Keyboard entry of symptoms and findings |
| **Part-picking robot** | % of parts in correct bins | Conveyor belt with parts, bins | Jointed arm and hand | Camera, joint-angle sensors |
| **Online shopping agent** | Price, quality, delivery time | Websites, vendors, shippers | Follow links, fill forms, display | Web pages (HTML text) |
| **Chess program** | Win / draw / loss | Board, opponent, clock | Move pieces | Board position |

# 7. The Nature of Environments

Environments differ along a few dimensions, and each dimension decides which kind of agent design works best.

| Property | Easy version | Hard version | Meaning |
|---|---|---|---|
| **Observability** | Fully observable | Partially observable | Do the sensors give the complete state of the world? |
| **Agents** | Single agent | Multi-agent | Is another agent (cooperative or competitive) present? |
| **Determinism** | Deterministic | Stochastic | Is the next state fully fixed by the current state and action? |
| **Episodes** | Episodic | Sequential | Does the current decision affect future decisions? |
| **Change** | Static | Dynamic | Does the world change while the agent deliberates? |
| **State values** | Discrete | Continuous | Finite number of states and actions, or a continuous range? |
| **Knowledge** | Known | Unknown | Does the agent know the 'rules' (outcomes of actions)? |

| Task environment | Observable | Agents | Deterministic | Episodic | Static | Discrete |
|---|---|---|---|---|---|---|
| Crossword puzzle | Fully | Single | Deterministic | Sequential | Static | Discrete |
| Chess with a clock | Fully | Multi | Deterministic | Sequential | Semi | Discrete |
| Poker | Partially | Multi | Stochastic | Sequential | Static | Discrete |
| Taxi driving | Partially | Multi | Stochastic | Sequential | Dynamic | Continuous |
| Medical diagnosis | Partially | Single | Stochastic | Sequential | Dynamic | Continuous |
| Image analysis | Fully | Single | Deterministic | Episodic | Semi | Continuous |

**TIP**
The **hardest** case is: partially observable, multi-agent, stochastic, sequential, dynamic, continuous and unknown. Real-world driving falls here.

# 8. The Structure of Agents

The **job of AI** is to design the agent program. A generic skeleton is: *receive a percept → update memory → choose an action → return it.* Five basic kinds of programs exist, in increasing order of generality.

## 8.1 Simple reflex agents

They choose actions using **only the current percept** and ignore history, using **condition-action (IF-THEN) rules**. Example: *IF car-in-front-is-braking THEN initiate-braking*. They are simple and fast, but work only if the environment is **fully observable**; otherwise they can loop forever.

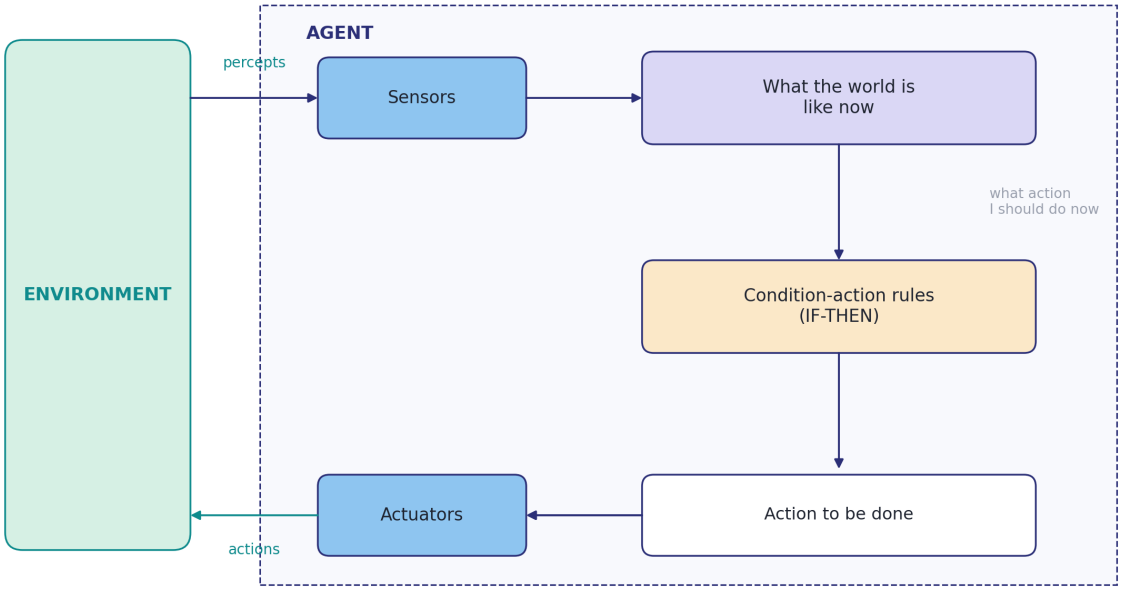



*Fig 1.6 Simple reflex agent*

## 8.2 Model-based reflex agents

To cope with **partial observability** the agent keeps an **internal state**, a model of how the world evolves and what its own actions do. It uses the model to guess the part of the world it cannot see.

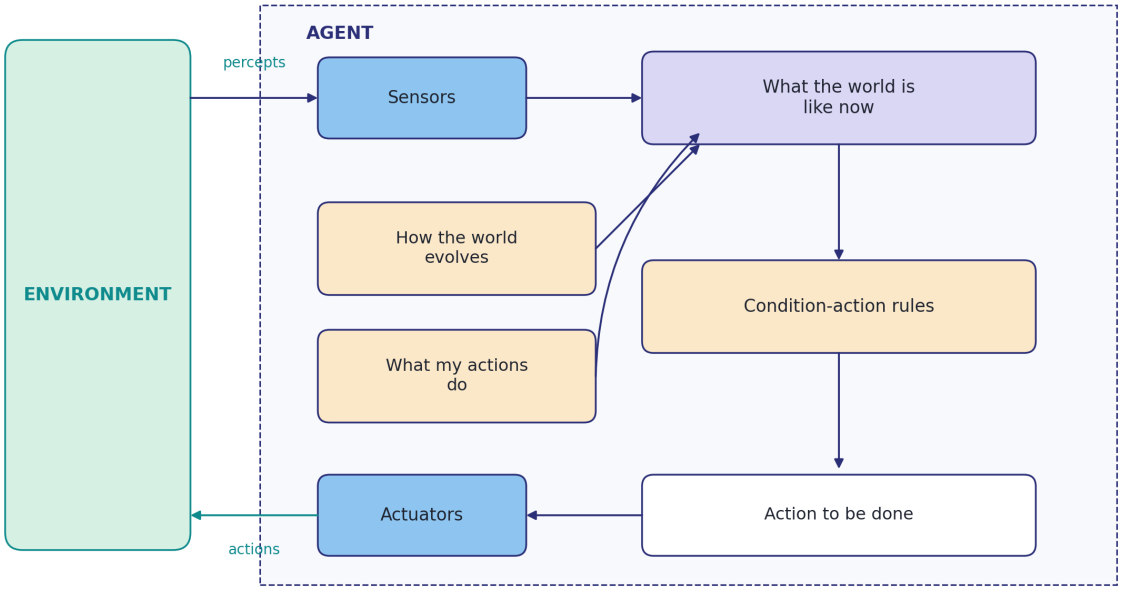



*Fig 1.7 Model-based reflex agent*

## 8.3 Goal-based agents

Knowing the current state is not enough; the agent also needs a **goal** describing a desirable situation. It considers 'What will happen if I do action A?' and chooses actions that reach the goal. This needs **search and planning**. It is more flexible: change the goal and the behaviour changes.

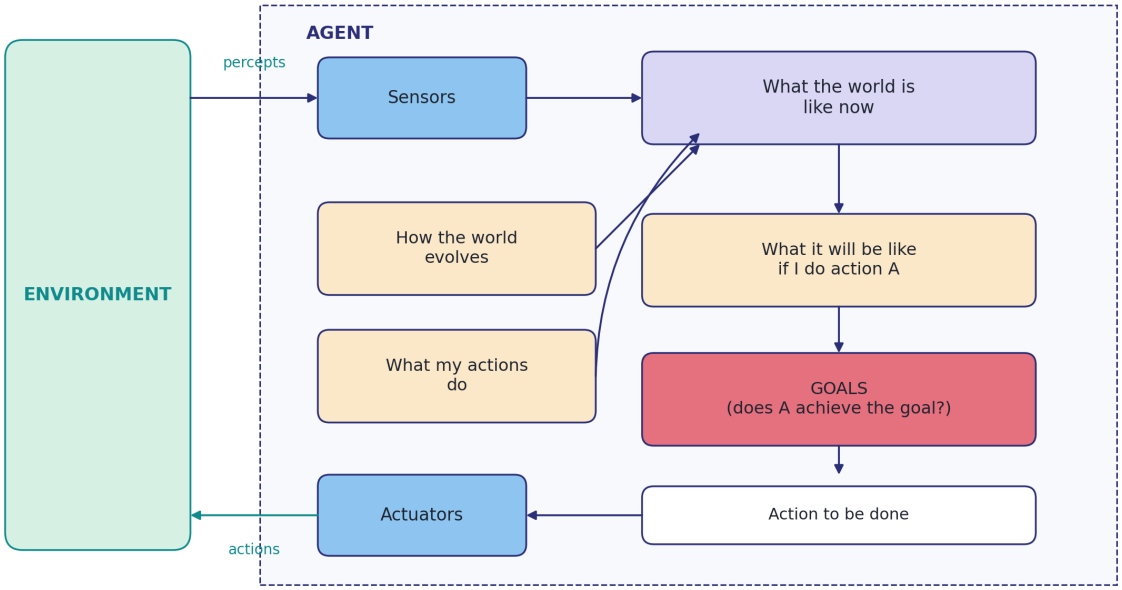



*Fig 1.8 Goal-based agent*

## 8.4 Utility-based agents

Goals give only a yes/no distinction (goal achieved or not). A **utility function** gives a numeric score of 'how happy' the agent is in a state, so it can choose the **best** route (fast *and* safe *and* cheap) and handle trade-offs and uncertainty by maximising expected utility.

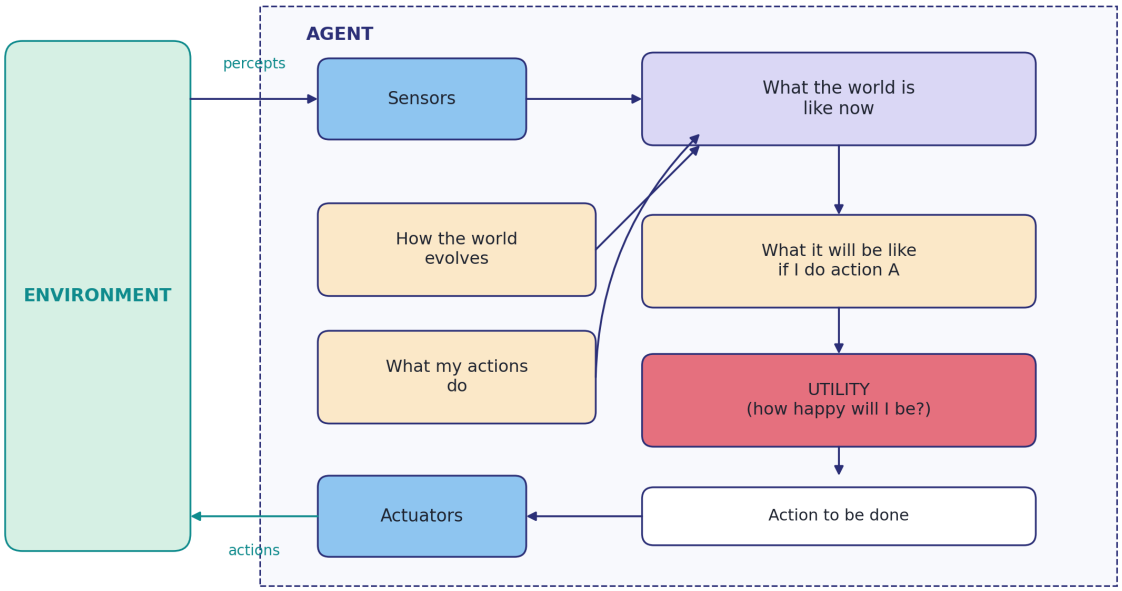



*Fig 1.9 Utility-based agent*

## 8.5 Learning agents

A learning agent can start in an unknown environment and become more competent. It has four conceptual components:

- **Performance element:** selects external actions (the agent we have described so far).
- **Learning element:** improves the performance element using feedback.
- **Critic:** tells the learning element how well the agent is doing against a fixed performance standard.
- **Problem generator:** suggests exploratory actions that lead to new, informative experiences.

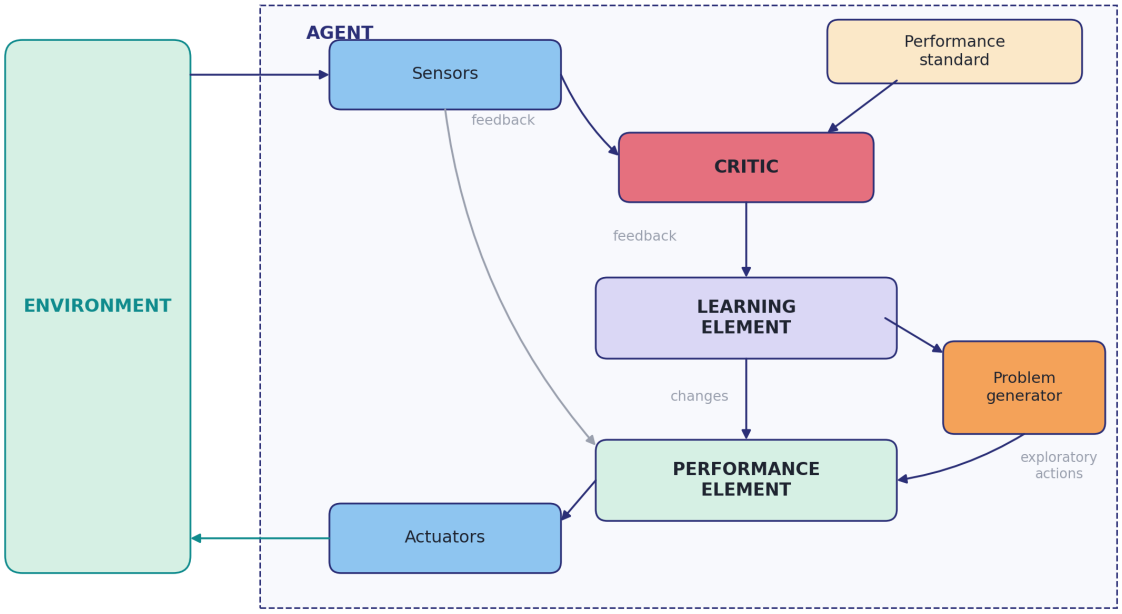



*Fig 1.9b General learning agent*

| Agent type | Uses | Strength | Limitation |
|---|---|---|---|
| **Simple reflex** | Current percept + rules | Fast, simple | Fails when partially observable |
| **Model-based** | Internal state + rules | Handles hidden information | Needs an accurate world model |
| **Goal-based** | Goals + search/planning | Flexible, purposeful | Cannot compare how good goals are |
| **Utility-based** | Utility function | Best trade-offs, handles uncertainty | Utility function is hard to define |
| **Learning** | Experience and feedback | Improves over time | Needs data and time |

## 9. Problem-Solving Agents

A **problem-solving agent** is a goal-based agent that decides what to do by **finding a sequence of actions that leads to a desirable state**. It works in four phases.

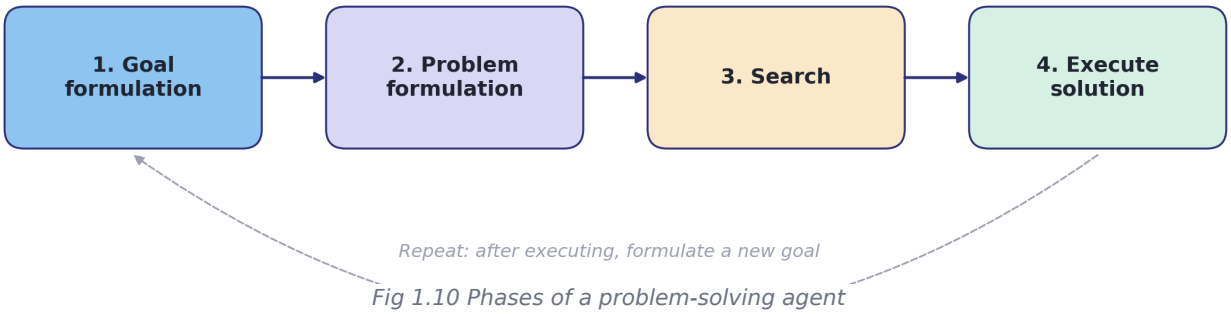



*Fig 1.10 Phases of a problem-solving agent*

- **Goal formulation:** decide which states count as success. This limits what the agent must consider.
- **Problem formulation:** decide which actions and states to consider (next section).
- **Search:** look for a sequence of actions that reaches the goal. The output is a **solution**.

- **Execution:** carry out the actions one by one. While executing, the agent can ignore its percepts (*open-loop* system) if the environment is observable, deterministic and known.

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
   state  <- UPDATE-STATE(state, percept)
   if seq is empty then
        goal    <- FORMULATE-GOAL(state)
        problem <- FORMULATE-PROBLEM(state, goal)
        seq     <- SEARCH(problem)
   action <- FIRST(seq);   seq <- REST(seq)
   return action
```

**EXAMPLE**
**Touring Romania.** An agent in Arad must reach Bucharest tomorrow. Goal: be in Bucharest. States: the cities. Actions: drive between adjacent cities. Search finds a route such as *Arad → Sibiu → Fagaras → Bucharest*; the agent then drives it.

# 10. Problem Formulation

Formulating a problem means defining the information the agent will use. A well-defined problem has **five components**:

| Component | Description | Romania route example |
|---|---|---|
| **1. Initial state** | Where the agent starts | In(Arad) |
| **2. Actions** | Possible actions in a state, ACTIONS(s) | Go(Sibiu), Go(Timisoara), Go(Zerind) |
| **3. Transition model** | RESULT(s, a): the state reached by doing action a in state s | RESULT(In(Arad), Go(Zerind)) = In(Zerind) |
| **4. Goal test** | Checks whether a state is a goal | Is the state In(Bucharest)? |
| **5. Path cost** | A numeric cost for a path (sum of step costs c(s, a, s')) | Total distance in km |

The **state space** is the set of all states reachable from the initial state. A **solution** is a path from the initial state to a goal state, and an **optimal solution** is the one with the lowest path cost.

**NOTE**
**Abstraction** matters: we remove irrelevant detail. 'Drive Arad → Zerind' hides weather, music and passengers. A good abstraction is valid (any abstract solution can be expanded into a real one) and useful (it is easier than the real problem).

## Example problems

**Toy problems** illustrate methods and are used to compare algorithms. **Real-world problems** are those people actually care about.

**(a) The 8-puzzle.** A 3×3 board has eight numbered tiles and a blank. A tile adjacent to the blank can slide into it.

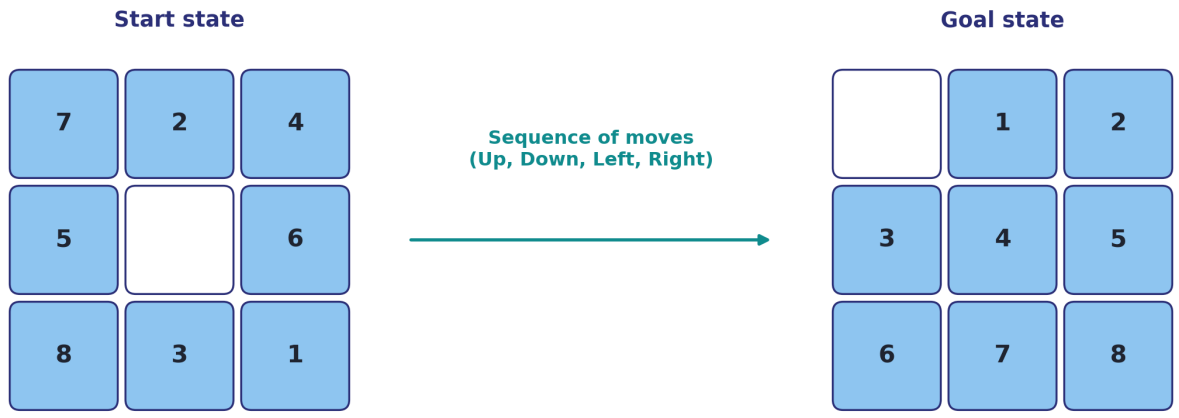



*Fig 1.11 The 8-puzzle: slide tiles into the blank space to reach the goal*

| Component | 8-puzzle |
|---|---|
| **States** | Location of each of the eight tiles and the blank (9!/2 = 181,440 reachable states) |
| **Initial state** | Any state (any given configuration) |
| **Actions** | Move the blank Left, Right, Up or Down |
| **Transition model** | Returns the resulting state, e.g. Left swaps the blank with the tile on its left |
| **Goal test** | State matches the goal configuration |
| **Path cost** | Each step costs 1, so cost = number of steps |

**(b) The vacuum world.** States: agent location and dirt in each square ($2 \times 2^2 = 8$ states). Actions: Left, Right, Suck. Goal: all squares clean. Path cost: 1 per action.

**(c) The 8-queens problem.** Place eight queens on a chessboard so that no queen attacks another. *Incremental formulation:* start with an empty board and add one queen at a time (states = any arrangement of 0-8 queens). A *complete-state formulation* starts with all eight queens and moves them. The better version only places queens in safe squares, cutting $1.8 \times 10^{14}$ states to just 2,057.

**(d) Real-world problems:** route finding (GPS navigation, airline travel planning), touring problems and the travelling salesperson problem (TSP), VLSI layout, robot navigation, automatic assembly sequencing and protein design.

| Toy problems | Real-world problems |
|---|---|
| Concise exact description | No single agreed description |
| Used to compare algorithms | People actually care about the answer |
| Examples: 8-puzzle, vacuum world, 8-queens | Examples: route finding, TSP, VLSI layout, robot navigation |

# Quick Revision: Unit I at a Glance

| Topic | One-line recall |
|---|---|
| **AI** | Building agents that act rationally to achieve goals |
| **Turing Test** | Judge cannot tell machine from human; needs NLP, knowledge, reasoning, learning |
| **Agent** | Perceives via sensors, acts via actuators; Agent = architecture + program |
| **Rational agent** | Maximises *expected* performance measure given its percept sequence and knowledge |
| **PEAS** | Performance, Environment, Actuators, Sensors |
| **Environment types** | Observable, agents, deterministic, episodic, static, discrete, known |
| **Agent types** | Simple reflex → model-based → goal-based → utility-based; learning agent improves any of them |
| **Problem formulation** | Initial state, actions, transition model, goal test, path cost |

## Frequently asked exam questions

- Define AI. Explain the four approaches to AI with a neat diagram.
- Explain the foundations of AI and briefly trace its history.
- What is a rational agent? Explain PEAS with at least two examples.
- Explain the different properties of task environments with examples.
- With neat diagrams, explain the five types of agent programs.
- Explain the five components of problem formulation using the 8-puzzle or vacuum world.